

PACE challenge 2024

DESTRADE Matthieu, DAGONNEAU Thomas

Abstract—In this paper, we explain our endeavor to tackle the PACE challenge of 2024, which centers on minimizing edge crossings in a two-layered drawing of a bipartite graph. The input comprises a bipartite graph with a total of $n_1 + n_2$ vertices. The linear order of the initial n_1 upper vertices remains constant and is provided as part of the input. However, the order of the n_2 bottom vertices can be modified. To address this challenge, we devised methodologies employing median computation, and a greedy approach.

I. INTRODUCTION

In this project, we tried to address the PACE challenge [2] of 2024, focusing on minimizing edge crossings in a bipartite graph's two-layered drawing. The challenge provided us with inputs categorized into three groups: tiny, medium, and large. Several tracks of research were proposed: we could try to compute an exact solution for medium instances, or an approximation for large ones. While the challenge offered the flexibility to tackle either aspect, we concentrated on the approximation problem.

In order to be able to test the heuristics we would create, we started by implementing ways of generating bipartite graphs with their solutions, so that we could test our algorithms on more examples than the ones provided by the PACE challenge. We therefore created an algorithm generating random bipartite graphs with a predetermined number of vertices, and another that brute-forces the optimal ordering for a graph by testing all the possible orders of vertices.

To tackle the minimization task, we then devised several solutions grounded in simple concepts. Initially, we explored how utilizing the median of the points could help us reduce the number of crossings. This involved identifying the central upper vertex among the vertices connected to a given bottom vertex. Finally, confronted with the limits of the previous method, we implemented a greedy algorithm wherein we systematically compared vertices

in pairs with exactness, subsequently ordering them in a greedy fashion.

With these methods, we got convincing results and managed to perform well better than a random configuration, and with very small computation times compared to a brute-force approach. However, we believe that our methods could still be improved a lot using more computation time, especially since the PACE challenge allows up to 5 minutes of computation for every graph.

All our code was made in Java using the JAR libraries and a code extension from Luca Castelli Aleardi [1]. Our code is accessible at <https://github.com/matthieuDESTRADE/Projet>

II. BACKGROUND

The one-sided crossing minimization problem (OCM) involves arranging the nodes of a bipartite graph on two layers (typically horizontal), with one of the layers fixed, aiming to minimize the number of edge crossings.

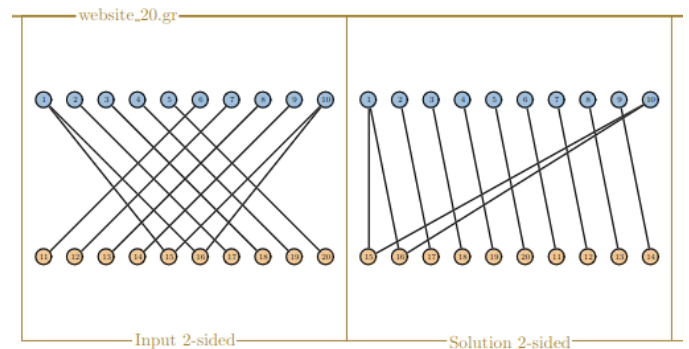


Fig. 1. Example of an OCM problem

OCM is one of the basic building blocks used for drawing hierarchical graphs. It is NP-complete [3], even for trees. This problem can't be solved in polynomial time (except if $P = NP$). Therefore, we need to implement approximation algorithms in order to be able to process large instances.

In this paper, we will use the following notations: G_1 is the upper part of the graph, and G_2 is the bottom one. n_1 is the number of vertices in G_1 and

n_2 is the number of vertices in G_2 . For v_1 and v_2 two vertices respectively from G_1 and G_2 , we note $e(v_1, v_2)$ the edge from v_1 to v_2 if it exists.

III. METHODOLOGY

In this section we will discuss the algorithms we designed.

A. Generation of graphs

In order to have a large quantity of graphs to test our heuristics, we created an algorithm that could generate new bipartite graphs.

It takes as input two integers, i_1 and i_2 . i_1 defines the value of $n_1 = n_2 = i_1$ for the generated graph. i_2 determines to what extent the graph will be dense. The algorithm iterates over every edge possible in the graph and creates it with a probability of $1/i_2$. It outputs the created graph in the same format as the ones given by the PACE challenge.

This method is implemented in the class `GenerateRandom`.

B. Brute-force approach

In order to be able to get the exact minimum number of crossings, at least for small graphs, we implemented a brute-force approach, that relies on two classes in our code: `Permutation` and `BruteForce`.

In the first class, `size` is given as input, and we compute and store the permutations of $[1, \text{size}]$ using element swapping and a recursive approach.

In the second one, we first represent the graph in a way that will allow us to apply permutations to the bottom vertices and count the number of crossings. For this we use two classes: `Node` and `Leaf`. A `Leaf` is only defined by an integer, its `id`, and a `Node` is defined by an `id` and a list of `Leafs` that are connected to it. Using these tools, we define our graph as a list of `Nodes` that correspond to the bottom vertices. This process takes $\mathcal{O}(n_1 \times n_2)$ time, and more precisely $\mathcal{O}(e)$ time where e is the number of edges in the graph.

We then change the order of the `Nodes` in the list using the `Permutation` class, and for each possibility we compute the number of crossings by iterating, for every edge, over all the other edges. We finally keep the best configuration. We return it

and print the number of crossings.

The computation of the permutations takes $\mathcal{O}(n_2!)$ time, and for every permutation the counting of crossings takes $\mathcal{O}(e^2)$. Therefore the complexity of the whole algorithm is $\mathcal{O}(n_2! \times e^2)$. It makes it impossible to use on large graphs. The maximum graph size with which we were able to compute a solution in reasonable time (a few seconds) is $n_1 = n_2 = 10$.

C. Median based organisation algorithm

Since the exact, brute-force algorithm was too slow, we tried a new algorithm that would run in almost linear time with the number of vertices, and based on the computation of medians. We had the idea to use medians because it seems that, in order to reduce crossings, the bottom vertices should be placed towards the way where they are connected to the most of upper vertices. It is implemented in the class `MedianHeuristic`.

The algorithm operates as follows: we build the graph as a list of `Nodes`, in a similar fashion to the previous method. Then, for each vertex v_2 in G_2 , we compute the median vertex of its leaves in G_1 . It is determined using the most centrally positioned vertex if the number of leaves is odd, or the mean between the two most centrally positioned vertices if the number of leaves is even. We then sort the vertices of G_1 according to the values of the medians.

The construction of the graph and the computation of the medians are done in $\mathcal{O}(e + n_2)$ time (since once the graph is built, we only have to iterate over the bottom nodes to compute the medians). The sorting of the nodes using efficient functions then takes $\mathcal{O}(n_2 \log(n_2))$ time ; therefore, the overall complexity of the algorithm is $\mathcal{O}(e + n_2 \log(n_2))$. This makes it very efficient on large graphs, but also inherently imprecise. In some cases, several nodes with the same median should be placed in the same order, and, even worse, sometimes the order of the medians is the opposite of the best order.

We tried variants of this heuristic, for instance using the means of leaves, but they were also very imprecise. This led us to try more precise methods.

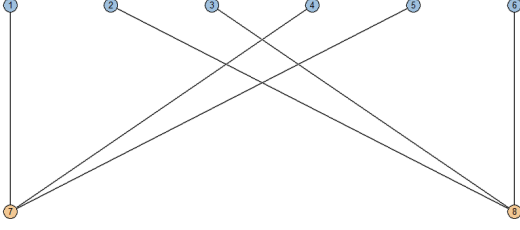


Fig. 2. Example of case where the median heuristic is wrong

D. Greedy algorithm

The last algorithm we implemented uses a greedy policy and is coded in the class `Heuristic`. After encoding the graph, we construct the order of the bottom vertices progressively. We add the first vertex, and then, for every unplaced vertex v_2 in G_2 , we compute with every vertex already placed the number of new intersections formed by adding v_2 to the right or the left of this vertex. We use it to compute for every potential position of v_2 the associated cost in terms of crossings. We then choose the position that minimizes the number of crossings created, and repeat this procedure until all vertices of G_2 have been placed. This is similar to an insertion sort.

It is a greedy algorithm because we make our decision at every step to optimize locally the result without regard to the next steps, and it is not exact because the order inferred by comparing the number of crossings between two vertices is actually not a real order and is not transitive. There are some scenarios with three vertices u , v , and w in which u should be placed left of v , v left of w , and w left of u , which is not possible. Nevertheless, it still efficiently orders vertices in a lot of cases.

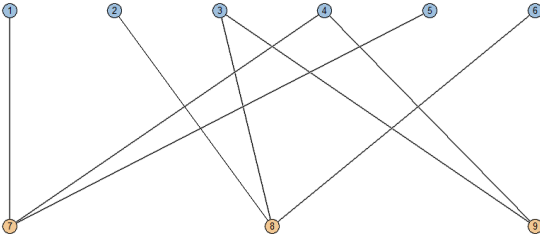


Fig. 3. Example of a case where the order is ambiguous

We implemented this method iteratively and

recursively. The advantage of the recursive way is that it allows us to easily explore several possibilities when adding a new vertex from G_2 , and to eventually keep the best order. For instance, we can try all the positions that have the same cost when placing a given vertex. However, with big graphs it rapidly becomes impossible to use as the maximum recursion depth is reached. We therefore implemented a way to switch from recursive to iterative after a certain number of tries. However, for big graphs, the number of tries we can make stays very limited.

With regard to complexity, the construction of the graph is still in $\mathcal{O}(e)$, and the iterative algorithm has to do $\mathcal{O}(n_2^2)$ times the computation of the number of intersections, which in the worst case is in $\mathcal{O}(n_1^2)$. In the worst case we therefore get a global complexity of $\mathcal{O}(e + n_1^2 \times n_2^2) = \mathcal{O}(e^2)$.

For the recursive algorithm, this number is multiplied by the number of tries we do. If we want to try every possibility, there are $n_2!$ tries to do (number of permutations of G_1), and the complexity becomes $\mathcal{O}(n_2! \times e^2)$, the one of the brute-force approach.

In practice, even the iterative method is far better than the median heuristic, even if it comes at a higher computational price. The recursive method could be adapted to finely select what configurations to try and further improve results.

IV. RESULTS AND DISCUSSION

In order to evaluate our heuristics we designed a simple scoring method. We generate several graphs of a chosen size and with a chosen density. We then compute the average of the errors for each graph. The error E is defined for a graph with the following formula :

$$E = \frac{c_h + 1}{c_{truth} + 1} - 1$$

where c_h is the number of crossings obtained with our heuristic, and c_{truth} the real minimum computed with the brute-force algorithm. This error should be close to 0 for a good heuristic.

The results with the computation times are the following :

We observe that our methods are very fast compared to the brute-force approach and yield

TABLE I

RESULTS FOR 10000 TRIES WITH GRAPHS OF SIZE $n_1 = n_2 = 7$ WHERE EVERY EDGE HAS A PROBABILITY $1/2$ OF EXISTING

Heuristic	Mean error	Computation time (ms)
Brute-force	0	5.49
Random (no sorting)	0.4842169	0.0026
Median	0.024016032	0.0045
Mean	0.004440446	0.0019
Greedy	3.758494E-4	0.0086

TABLE II

RESULTS FOR 10000 TRIES WITH GRAPHS OF SIZE $n_1 = n_2 = 7$ WHERE EVERY EDGE HAS A PROBABILITY $1/3$ OF EXISTING

Heuristic	Mean error	Computation time (ms)
Brute-force	0	3.6116
Random (no sorting)	0.9572045	0.0011
Median	0.02506076	0.0047
Mean	0.009588264	0.002
Greedy	0.0010431799	0.0063

TABLE III

RESULTS FOR 10000 TRIES WITH GRAPHS OF SIZE $n_1 = n_2 = 7$ WHERE EVERY EDGE HAS A PROBABILITY $1/5$ OF EXISTING

Heuristic	Mean error	Computation time (ms)
Brute-force	0	2.36
Random (no sorting)	1.9862305	0.0021
Median	0.012991054	0.004
Mean	0.009067744	0.0024
Greedy	8.715944E-4	0.006

TABLE IV

RESULTS FOR 50 TRIES WITH GRAPHS OF SIZE $n_1 = n_2 = 10$ WHERE EVERY EDGE HAS A PROBABILITY $1/3$ OF EXISTING

Heuristic	Mean error	Computation time (ms)
Brute-force	0	7849.86
Random (no sorting)	0.5361329	0.02
Median	0.033782214	0.14
Mean	0.010316407	0.02
Greedy	0.0018208527	0.0

very convincing results compared to what a random order would give, especially the greedy one, which is 10 times better than the median and mean heuristics. Their performances are also fairly the same over all the sizes and densities of graphs we tested.

It is also interesting to see how all the methods compare to each other for bigger graphs, even if we do not have the real solution:

TABLE V

RESULTS FOR THE GRAPH 25.GR IN THE HEURISTIC-PUBLIC DATABASE

Heuristic	Number of crossings	Computation time (ms)
No sorting	448841	0.0
Median	270958	50
Mean	272176	44
Greedy	265323	6513

With bigger graphs, the heavier computational

TABLE VI

RESULTS FOR THE GRAPH 50.GR IN THE HEURISTIC-PUBLIC DATABASE

Heuristic	Number of crossings	Computation time (ms)
Random (no sorting)	1553	0.0
Median	1552	5
Mean	1751	6
Greedy	1552	326

TABLE VII

RESULTS FOR THE GRAPH 75.GR IN THE HEURISTIC-PUBLIC DATABASE

Heuristic	Number of crossings	Computation time (ms)
Random (no sorting)	9319457	0.0
Median	5579909	21
Mean	5573430	6
Greedy	5462173	850

weight of the greedy method compared to the other ones becomes evident, but it does come with better results. All the methods give a far better result than the one we would get without sorting. It is also interesting to note that the mean heuristic tends to perform worse than the median one on real examples, whereas it was the opposite with the generated graphs.

For all heuristics, we did not take the time required to compute the number of crossings into account if it was not directly used by the method to compute its result.

V. CONCLUSIONS

To conclude, in this project, we designed several algorithms to tackle the one-sided crossing minimization problem (OCM). We tried to design fast algorithms that would yield good approximations of the optimal result. We managed to do it using two heuristics : the first one uses medians (or means) of clusters of nodes to compute an order, and the other determines this order greedily by computing the cost of iteratively placing each bottom node among the other ones.

The PACE challenge allows us to use 5 minutes of computation time for each graph. Since our algorithms are usually a lot faster than this on the provided examples, we have a lot of room for improvement and could still enhance our heuristics, especially the greedy one. Using its recursive version, we have the tools to find better configurations. We could use thresholds to control the maximum recursion depth and the number of possibilities we allow the algorithm to explore in order to increase our chances of getting the right solution. This makes several new questions arise, and especially how

to determine if a configuration should be explored further or not.

However, we should mention that some of the examples provided by the PACE challenge (especially `44.gr` and `10.gr` for which we had to use the median heuristic) contain far more vertices and edges than the others, and that our greedy algorithm struggles to compute an order for them. It might be possible in our code to add a threshold to switch between the methods used and use simpler heuristics if the graphs are too big.

REFERENCES

- [1] Luca Castelli Aleardi, *INF562 - Géométrie algorithmique de la théorie aux applications*
- [2] PACE. *PACE 2024*, <https://pacechallenge.org/2024/>, accessed 20 March 2024.
- [3] Peter Eades, Sue Whitesides. *Drawing graphs in two layers*, <https://www.sciencedirect.com/science/article/pii/0304397594901791?via%3DiHub>, accessed 20 March 2024.